Freight Rate Prediction

Data validation and train/test split report | Machine Learning Engineer assessment

The task is to predict **posted_rate** for 12,000 loads in Nov-Dec 2025, using 48,000 labeled loads from Jan-Oct 2025. The final model is a structural pricing model on log(rate). In a backtest built to mirror the validation task, it reaches **1.24% mean absolute percentage error (MAPE)** on clean labels with no systematic bias, against 1.75% for LightGBM with the same engineered features.

1.24%	**$29**	**1.41%**	**0.07%**
expected MAPE, clean labels	expected mean absolute error	MAPE on loads touching an unseen city	largest bias in any fold

1. Validating the data

I profiled every column in both files and tested each repair against the labeled data. Bad values are two to three times more common in validation, so these repairs matter even more there.

Issue	Training	Validation	Treatment	Evidence
Corrupted posted_rate	677 (1.41%)	Unknown	Removed from training by model residual	Off by x0.18-0.44 or x2.3-5.3. In log terms, clean loads sit within 0.08 of the model and corrupted ones at least 0.81 away, with nothing between
Negative weight	292 (0.61%)	145 (1.21%)	Absolute value	Sign errors. Price error once flipped is 0.0141, like normal loads (0.0137). A median fill doubles it (0.0286)
Missing weight	300 (0.62%)	165 (1.38%)	Training median	Unbiased but less precise: price error 0.0281
Missing market_index	374 (0.78%)	249 (2.08%)	Same-day average across loads	The index varies by 0.025 within a day and 0.17 between days. Price error 0.0158
Unseen cities	None	8 cities, 1,447 (12.06%)	Priced from coordinates	City effects form a smooth map. Backtested MAPE is 1.41%, against 1.21% for known cities
quote_signal	All rows	All rows	Excluded	Unstable, and unrelated to price in Nov-Dec. See section 2

No problems found: 0 duplicate load IDs, no duplicate rows, one date format, and one coordinate pair per city. Coordinates do not match real-world locations (Los Angeles sits at latitude 28.6), but **distance** agrees with them at a median 1.18 times the straight line, so I used them as supplied.

2. The quote_signal trap

quote_signal looks like the best feature in the file and is the most dangerous one. It tracks rate per mile almost perfectly in some months, is almost perfectly inverted in others, and is unrelated in August. Nov-Dec has no labels, so I compared it with predictions from a model that never sees it. The correlation there is 0.01, which means a model trained on the relationship would be relying on one that no longer exists.

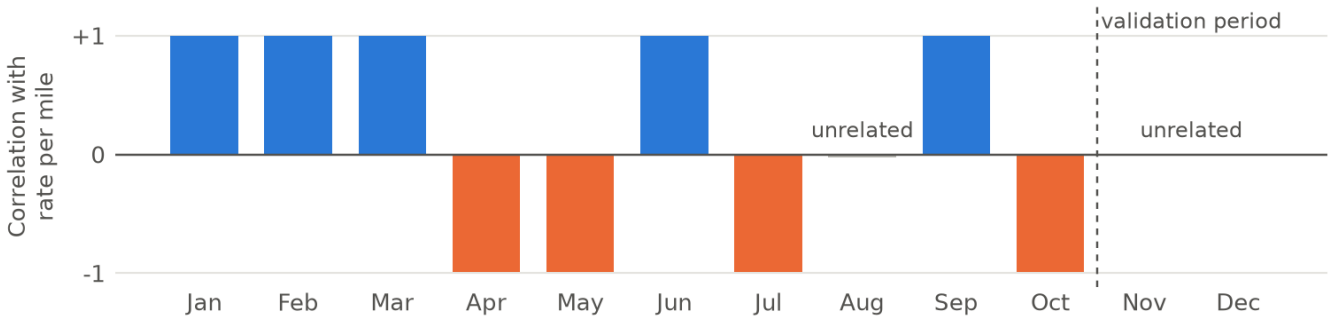


Figure 1. Rank correlation with rate per mile. Jan-Oct uses actual rates. Nov-Dec uses model predictions, as no labels exist.

LightGBM test, clean labels	MAPE with quote_signal	MAPE without
Random 80/20 split	1.08%	1.51%
Train Jan-Aug, test Sep-Oct	2.30%	2.53%
Train Jan-Jul, test Aug (signal unrelated to price, as in Nov-Dec)	**5.37%**	**1.43%**

A random split recommends keeping the feature, because every month's relationship is in training. It helps slightly on Sep-Oct, where it is still tied to price, but that link is gone by Nov-Dec. The August test shows the real cost. On the validation file itself, adding quote_signal moves predictions by 3.7% on average with no information behind the move.

3. How I split the data

The validation file differs from the training data in three ways, so the test design copies all three. A random split would copy none of them.

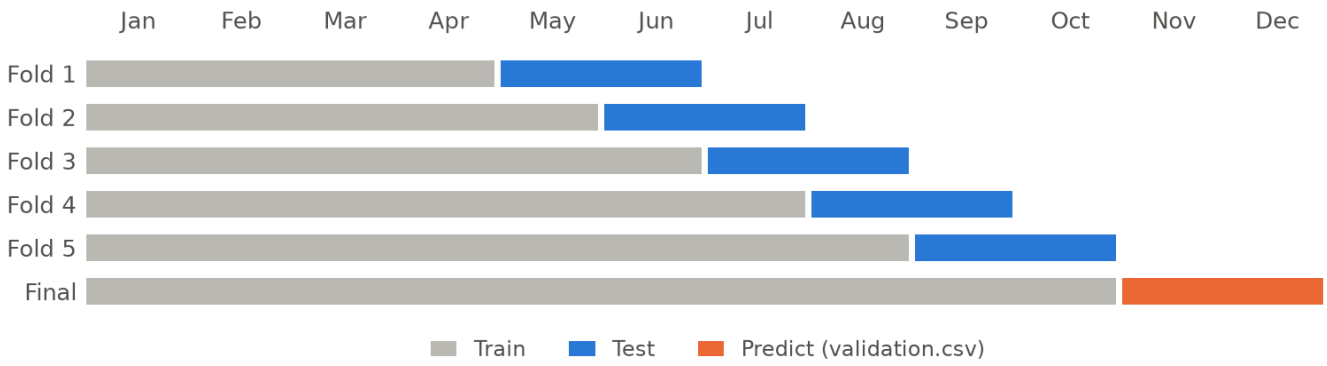


Figure 2. Five rolling folds. Each trains on the past and tests on the next two months, the same horizon as Nov-Dec.

Future dates. Every fold trains only on earlier months and tests on the next two. The folds cover both kinds of two-month window that matter: those with a quarter-end ramp and those without.

Unseen cities. 12.1% of validation loads touch a city with no training history. In each of five repeats per fold, I removed 8 random cities from training and scored test loads touching them separately. Final figures are weighted to the validation mix of known and unseen cities.

Corrupted labels. Each fold removes them from its own training rows only. I can't tell whether Spotter's answer key contains them, so I report results both ways.

Leakage. Weight fills and model transforms are learned on training rows. The same-day market average and step detection read feature values only, never a label.

How blind these results are. I chose the first model design on folds 1-3. When I then ran folds 4-5, they came in low by as much as 1.2%. The cause was three abrupt steps in market_index (Apr 16: +0.11, Jul 25: -0.17, Oct 3: +0.13), where prices react more than they do to ordinary index moves. Detecting those steps from the index and adding one coefficient brought bias below 0.1% in every fold, including folds that meet a step they never trained on. Because I changed the model after seeing those folds, and had already explored all ten months, these results are not fully blind. Read them as a strong estimate rather than a pristine holdout.

4. Results

Model	MAPE	MAE	RMSE	Bias
Structural (final)	1.24%	$29	$42	-0.01%
Structural + LightGBM on residuals	1.23%	$29	$42	-0.01%
LightGBM, engineered features	1.75%	$41	$60	-0.43%
LightGBM, raw features	2.39%	$57	$85	-1.17%
Lane median	6.88%	$163	$247	-2.85%

Clean labels, averaged over five folds and weighted to the validation mix.

Rates here are close to additive on a log scale with Gaussian noise, so stating that form directly works better than asking trees to rediscover it. It also suits this task: trees hold their last value flat beyond the training dates, so they cannot carry the upward trend into Nov-Dec, and that shows up as their bias. Adding LightGBM on the residuals gains only 0.01 points, which does not justify a second model. Each part of the final model earns its place. MAPE rises to 1.40% without lane effects, 1.40% without market steps, 2.53% without the quarter-end ramp, 2.72% without the trend.

Fold	Test window	MAPE, known cities	MAPE, unseen cities	Bias, final model
1	May-Jun	1.24%	1.41%	-0.07%
2	Jun-Jul	1.24%	1.42%	+0.07%
3	Jul-Aug	1.20%	1.40%	+0.05%
4	Aug-Sep	1.19%	1.41%	-0.02%
5	Sep-Oct	1.20%	1.42%	0.00%

If the answer key contains corrupted labels. Results above exclude them. If Spotter's labels carry the same 1.4% error rate, expect about 3.5% MAPE, $84 MAE, $626 RMSE and R-squared of 0.83. No model can predict random errors of that size, so I predict the true price rather than inflating predictions to chase them.

What the model learned. Reefer costs 12.5% and Flatbed 7.5% more than Dry Van. Rates drift up about 7% a year. A 0.1 rise in market_index adds about 1.2%, or 2.0% when it arrives as an abrupt step. Each lane has its own stable premium or discount, typically about 1%, learned with shrinkage so that lanes with few loads stay close to zero.

5. The December prediction chart

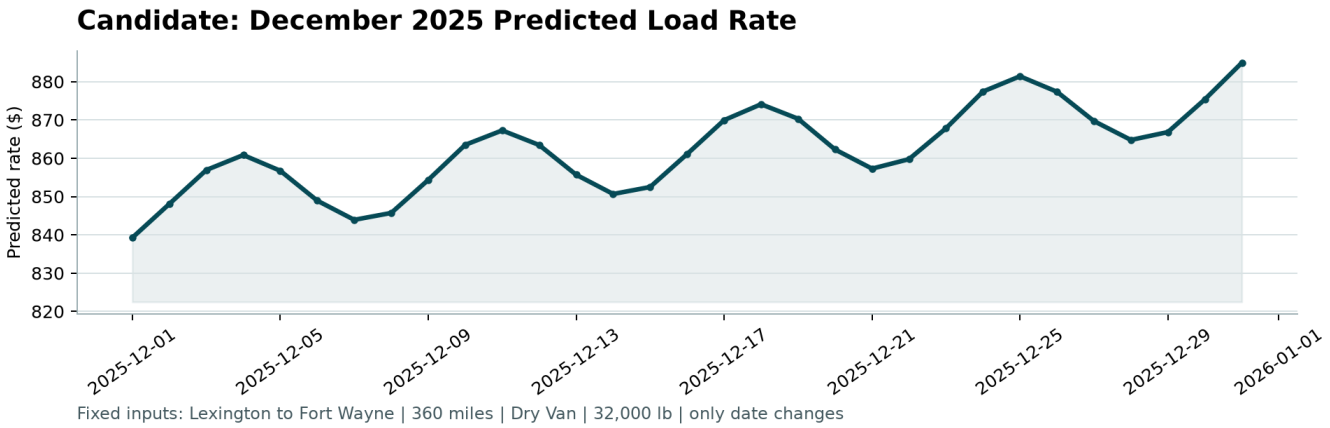


Figure 3. Output of the provided score.py, unmodified.

Only the date changes, so this chart isolates the model's calendar behavior. The December file has no market_index, so I used each date's average across validation.csv loads. Three effects shape the line:

Weekly cycle. The market index peaks Wednesday to Thursday and bottoms out Sunday to Monday.

Quarter-end ramp. Rates climb through the last month of every quarter and reset on the 1st. March, June and September show the same pattern at the same size (Figure 4). A Dry Van load gains about 2.5% by the 31st.

Trend. The steady upward drift adds about 0.6% over the month. Together, these put the last week 2.8% above the first on average, and the line runs from $839 on Dec 1 to $885 on Dec 31.

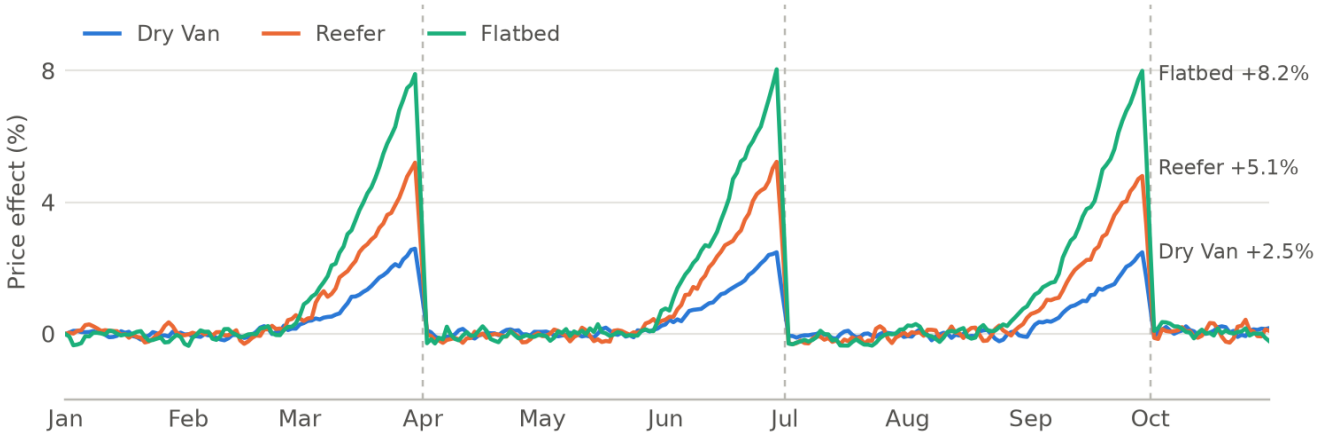


Figure 4. Quarter-end ramp by equipment, with all other model effects removed (3-day average). Dashed lines mark quarter starts.

6. Limits

No year-end history. The December ramp comes from three earlier quarter-ends. The eight federal holidays in the training data all sit within 0.3% of the model with no consistent direction, so I added no holiday effect, but the data has no Thanksgiving or Christmas. A real peak-season effect would not appear in these predictions.

New market shocks. market_index shows no step in Nov-Dec, so none is applied. The model can react to a step visible in the index, but not to a price change that does not appear there.

Unseen cities. A new lane's own premium cannot be known, which is why these loads run about 0.2 points higher in error.